

USER STORY • DXV-FRE-005

Emit the correct acquisition success / error envelope flag per run

Epic: Feedback Flags & Envelope Semantics | DXV Feedback (Quarantine Export)

Story Key	DXV-FRE-005	Priority	Highest
Epic	Feedback Flags & Envelope Semantics	Story Points	5
Components	failed-record-export	Issue Type	Story
Labels	dxv, failed-record-export, flags, envelope, integration	Status	Ready for Test

User Story

As a file-based automation owner

I want exactly one envelope flag file per run that states whether the acquisition succeeded cleanly or had validation failures

so that downstream DXV file-based automation can trigger the correct branch (success vs error handling) from a single, predictable trigger file.

Description

Every run produces exactly one envelope flag. When NO quarantine rows were exported (i.e. no table had failures), the stage writes `acq_success_{YYYYMMDD}.zip.flag`. When ANY table had failures, it instead writes `envelope_error_{YYYYMMDD}.zip.flag`. The two are mutually exclusive — one flag per run — using DXV-compatible naming.

The `envelope_error` flag body is text, with one failure summary per line in the form: `{table_name}_{DDMMYYYY}, {reason}, {count}`. The first column is the logical table name plus the processing date in DDMMYYYY (e.g. `customer_13032026` for 13 March 2026). The reason comes from the validation result (`error_type` and `field_name`, space-separated). If `validation_errors` cannot be exploded for a table, a single fallback line uses the row count instead.

The `acq_success` flag body is a single newline, kept that way for DXV-style trigger compatibility. The exact payload should be treated as part of the integration contract with downstream pickup. Both flags are written under the feedback root for the date.

Key Terms & Behaviour

acq_success flag	<code>acq_success_{YYYYMMDD}.zip.flag</code> — written when NO quarantine rows exported.
envelope_error flag	<code>envelope_error_{YYYYMMDD}.zip.flag</code> — written when ANY table had failures.
One per run	The two flags are mutually exclusive; exactly one is emitted.
error line format	<code>{table_name}_{DDMMYYYY}, {reason}, {count}</code>

Date in line	DDMMYYYY (e.g. customer_13032026), distinct from the YYYYMMDD in the filename.
Reason	error_type and field_name, space-separated; fallback line uses count when errors cannot be exploded.
acq_success body	A single newline (exact payload is part of the integration contract).

Acceptance Criteria

Definition of Done

1. A run with zero exported quarantine rows writes acq_success_{YYYYMMDD}.zip.flag and no envelope_error flag.
2. A run where at least one table had failures writes envelope_error_{YYYYMMDD}.zip.flag and no acq_success flag.
3. Never are both flags present for the same run.
4. Each envelope_error line is '{table_name}_{DDMMYYYY}, {reason}, {count}' with the date in DDMMYYYY.
5. A table whose errors cannot be exploded contributes a single fallback line using the row count.
6. The acq_success flag body is exactly a single newline.
7. Flag filename uses YYYYMMDD while the in-body first column uses DDMMYYYY.

Recommended Test Data

process_date	20260313 (filename uses 20260313)
In-body date	13032026 (DDMMYYYY for 13 March 2026)
Success flag name	acq_success_20260313.zip.flag
Error flag name	envelope_error_20260313.zip.flag
Error line example	customer_13032026, null_check customer_id, 12
Fallback line example	account_13032026, 7

Global Preconditions

Environment & Setup

- Quarantine sources are seeded to match the success/error scenario under test.
- The feedback root key prefix for the date is known so the flag can be located.

Test Cases

Coverage: 6 test case(s) Happy Path x2 Boundary x3 Negative x1

TC-001 Happy path — acq_success when no failures

HAPPY PATH **PRIORITY: HIGHEST**

Objective: A clean run writes only the acquisition-success flag.

#	Test Step / Action	Test Data (format / value)	Expected Result
1	Ensure no quarantine rows will be exported.	No quarantine tables / all empty for 20260313	No failures present.
2	Run the export.	process_date=20260313	Run completes.
3	Locate the flag under the feedback root.	landing/acquisition/feedback/20260313/	acq_success_20260313.zip.flag exists; envelope_error_20260313.zip.flag does NOT.
4	Inspect the success flag body.	Read object bytes	Body is exactly a single newline character.

TC-002 Happy path — envelope_error when failures exist

HAPPY PATH **PRIORITY: HIGHEST**

Objective: A run with failures writes only the envelope-error flag with correct line format.

#	Test Step / Action	Test Data (format / value)	Expected Result
1	Seed a table with explodable errors.	customer: 12 rows null_check/customer_id	Quarantine populated.
2	Run the export.	process_date=20260313	Run completes.
3	Locate the flag.	landing/acquisition/feedback/20260313/	envelope_error_20260313.zip.flag exists; acq_success does NOT.
4	Inspect the error line.	Read object text	Line: 'customer_13032026, null_check customer_id, 12' (DDMMYYYY date; reason space-separated; correct count).

TC-003 Boundary — multiple tables produce multiple lines

BOUNDARY **PRIORITY: HIGH**

Objective: Each failing table contributes its own summary line(s) to one envelope_error flag.

#	Test Step / Action	Test Data (format / value)	Expected Result
1	Seed multiple failing tables.	customer (12 null_check/customer_id), transaction (4 range/amount)	Two tables populated.
2	Run the export.	process_date=20260313	Run completes.
3	Inspect the flag body lines.	Read object text	Contains 'customer_13032026, null_check customer_id, 12' and 'transaction_13032026, range amount, 4'.

TC-004 Boundary — fallback line when errors cannot be exploded

BOUNDARY **PRIORITY: HIGH**

Objective: A table with empty/null errors contributes a single fallback line using the row count.

#	Test Step / Action	Test Data (format / value)	Expected Result
---	--------------------	----------------------------	-----------------

1	Seed a failing table whose errors are all empty/null.	account: 7 rows, validation_errors = [] / null	Quarantine populated; not explodable.
2	Run the export.	process_date=20260313	Run completes; this counts as a failure for envelope purposes.
3	Inspect the fallback line.	Read object text	Single fallback line for account uses the row count, e.g. 'account_13032026, 7' (count instead of reason).

TC-005 Negative — never both flags for one run**NEGATIVE** **PRIORITY: HIGHEST****Objective:** *The two flags are mutually exclusive in all scenarios.*

#	Test Step / Action	Test Data (format / value)	Expected Result
1	Run a mixed scenario (some failures, some clean tables).	customer has failures; account is clean/missing	Run completes.
2	Enumerate flag objects for the date.	List landing/acquisition/feedback/20260313/*.flag	Exactly one flag present (envelope_error, because at least one table failed); acq_success absent.

TC-006 Boundary — date encoding split (filename vs body)**BOUNDARY** **PRIORITY: MEDIUM****Objective:** *Filename uses YYYYMMDD while the in-body first column uses DDMMYYYY.*

#	Test Step / Action	Test Data (format / value)	Expected Result
1	Run with a date whose digit pattern makes the two formats obviously different.	process_date=20260901 (1 Sep 2026 -> body 01092026)	Run completes with failures present.
2	Compare filename and body date encodings.	Filename vs first column	Filename: envelope_error_20260901.zip.flag; body column: <table>_01092026 — encodings differ as documented.

Generated for the DXV Feedback (Quarantine Export) integration test suite — 2026-06-11.